

1 ĐẾM ĐƯỜNG THẲNG

Ta dễ dàng tính được hệ số góc của đường thẳng đi qua 2 điểm (x_1, y_1) và (x_2, y_2) là $T = \frac{y_2 - y_1}{x_2 - x_1} = \frac{u}{v}$.

Để chuẩn hoá phân số, ta phải đảm bảo u là số nguyên, v là số nguyên dương và $\gcd(u, v) = 1$.

Lưu ý: Ta cần xét riêng trường hợp $u = 0$ và trường hợp $v = 0$.

Sau khi tính được các số T_i tương ứng cho từng đường thẳng, số lượng các phần tử T_i phân biệt chính là đáp án của bài toán.

Việc tìm số lượng các phần tử T_i phân biệt có thể được thực hiện bằng các hàm cơ bản của thư viện **algorithm** là **sort** và **unique** hoặc cấu trúc dữ liệu **set**.

Độ phức tạp: $\mathcal{O}(n \cdot \log n)$

2 LÀM CHO LIÊN THÔNG

Xét trường hợp $n \leq 100$:

Ta có thể nối tất cả cặp đỉnh (u, v) của đồ thị với chi phí $a_u + a_v$.

Dễ thấy nếu ta nối theo cách này sẽ sinh ra $\frac{n(n-1)}{2}$ cạnh, kết hợp với m cạnh đặc biệt cho thêm là $T = \frac{n(n-1)}{2} + m$ cạnh.

Ta dễ dàng tìm ra cây khung cực đại trên đồ thị có n đỉnh và T cạnh này bằng thuật toán Prim hoặc Kruskal.

Độ phức tạp: $\mathcal{O}(T \cdot \log T)$

Xét trường hợp $n \leq 1000$:

Xét đỉnh x , ta nhìn đỉnh x dưới góc độ tham lam của thuật toán tìm cây khung nhỏ nhất:

- Nếu ta chọn cách nối đỉnh x với các đỉnh khác thông qua các cạnh đặc biệt, theo thuật toán tham lam, ta sẽ chọn cạnh có trọng số nhỏ nhất.

- Nếu ta không có các cạnh đặc biệt hoặc không dùng đến các cạnh đặc biệt, ta sẽ nối đỉnh x với một đỉnh y nào đó với chi phí $a_x + a_y$, theo thuật toán tham lam, ta sẽ chọn đỉnh y sao cho a_y nhỏ nhất.

Nhận xét: Xét đỉnh x , nếu ta thực hiện việc nối đỉnh x với các đỉnh khác mà **không** dùng đến các cạnh đặc biệt, ta chỉ cần quan tâm đến đỉnh $y \neq x$ mà a_y là nhỏ nhất.

Từ nhận xét trên, ta chỉ cần tìm cây khung nhỏ nhất trên đồ thị có $n + m$ đỉnh.

Với $n \leq 1000$, ta có thể sử dụng thuật toán Prim ở dạng chưa tối ưu.

Độ phức tạp: $\mathcal{O}(n + m)^2$

Xét trường hợp $n \leq 2 \cdot 10^5$:

Ta xử lý giống trường hợp $n \leq 1000$, tuy nhiên cần sử dụng thuật toán Kruskal hoặc thuật toán Prim kết hợp cấu trúc dữ liệu **heap**.

Độ phức tạp: $\mathcal{O}[(n + m) \cdot \log(n + m)]$

3 NGƯỜI HÂM MỘ TRUNG THÀNH

Xét trường hợp $k_i = 1$:

Đây là một bài toán quy hoạch động kinh điển, tên là bài toán cái túi (knapsack problem).

Độ phức tạp $\mathcal{O}(n \cdot x)$

Xét trường hợp $k_i = 4$:

Do k_i đủ nhỏ, ta có thể tách mỗi sản phẩm thành k_i sản phẩm lẻ, sau đó giải như bài toán cái túi thông thường.

Độ phức tạp $\mathcal{O}(n \cdot k_i \cdot x)$

Xét trường hợp $k_i \leq 1000$:

Do lúc này k_i đã khá lớn, việc ta tách mỗi sản phẩm thành k_i sản phẩm lẻ là điều không khả thi.

Lưu ý: Xem trước bài **Missing Coin Sum**.

Ta sẽ tiến hành tách khéo léo hơn, mỗi sản phẩm ta sẽ tách tối đa thành $m_i = \log k_i$ sản phẩm khác nhau sao cho từ m_i sản phẩm đó, ta luôn có thể lấy được một số lượng bất kỳ từ 1 đến k_i sản phẩm.

Tính đúng đắn của cách tách sau đây có thể chứng minh được thông qua cách giải bài **Missing Coin Sum** nêu trên.

```
vector <pair<int ,int>> items ;
void Add(int h, int s, int k)
{
    if (k == 1)
    {
        items.push_back({h,s});
    }
    else
    {
        int v = k/2;
        items.push_back({v*h,v*s});
        Add(h,s,k-v);
    }
}
```

Sau khi tiến hành tách xong, ta sẽ có tối đa $n \cdot \log k_i$ phần tử, trong giới hạn có thể áp dụng bài toán cái túi.

Độ phức tạp: $\mathcal{O}(n \cdot \log k_i \cdot x)$